

Certamen Artificialis Intelligentia: Evaluating AI in solving AI-generated programming exercises

No Author Given

No Institute Given

Abstract. Large language models (LLMs) are transforming programming education by enabling automated generation and evaluation of coding exercises. While previous studies have evaluated LLMs' capabilities in one of these tasks, none have explored their effectiveness in solving programming exercises generated by other LLMs. This paper fills that gap by examining how state-of-the-art LLMs—ChatGPT, DeepSeek, Qwen, and Gemini—perform when solving exercises generated by different LLMs. Our study introduces an original evaluation methodology that includes custom prompts and coding exercises in three commonly used programming languages: Python, Java, and JavaScript. The results have both practical and theoretical value. Practically, they help identify which models are better at generating and solving exercises produced by other LLMs. Theoretically, the study analyzes how solving exercises generated by LLMs compares to solving human-written ones. This contributes to understanding the role of LLMs not only as learners or tutors but also as collaborators in educational content creation.

Keywords: Large Language Models · Gamified Programming Exercises · AI-driven Assessment · Automated Code Evaluation.

1 Introduction

Using large language models (LLM) such as ChatGPT, DeepSeek, Qwen, or Gemini, represents a significant advance in programming education and skill assessment [11]. The state-of-the-art models, trained on extensive corpora of text and source code, can generate programming exercises that align with specific pedagogical objectives, ranging from fundamental syntax practice to advanced algorithmic problem solving, for various programming languages [13]. The underlying mechanisms of LLM enable them to synthesize relevant and varied coding challenges by using probabilistic predictions and contextual embeddings [6]. Such a capability not only supports the creation of customized learning materials but also facilitates the exploration of novel problem domains, thereby enhancing the adaptability and scalability of educational resources [30], even automatic programming use cases [18].

In addition to programming exercise generation, LLMs demonstrate promising performance in their solving [8, 10, 29, 27] and the evaluation of their solutions [17]. The models can employ static code analysis to identify semantic and syntactic errors [20, 7], generate test cases to validate functional correctness and

simulate edge-case scenarios to assess the robustness of the solution [5]. In addition, they can provide qualitative feedback on aspects such as computational efficiency, adherence to coding standards, and algorithmic design principles [33]. Integrating LLMs into programming education introduces several methodological advantages, including enhanced personalization, scalability, and accessibility. Applied AI allows personalized learning paths [28], and dynamic adjustment of the complexity and focus of exercises based on individual proficiency levels, as inferred through interaction patterns and performance metrics. Scalability is achieved by automating the exercise generation and evaluation [25], enabling teaching large cohorts without compromising quality. Moreover, the accessibility of LLM-powered platforms democratizes access to high-quality programming education, particularly for learners in resource-constrained environments. However, potential limitations exist to the proliferation of LLMs, such as training data biases, model output variability, and the risk of over-reliance on automated systems [14].

This study contributes to research on using LLMs in programming education by investigating their effectiveness in solving coding exercises generated by competitive LLMs. After this introduction section outlines the role of LLMs in generating, solving and evaluating programming exercises, the paper is further structured as follows. Section 2 reviews the state-of-the-art in AI-driven programming education, covering code generation, automated assessment methods, and challenges. Section 3 deals with materials and methods describing the experimental setup, including the selection of LLMs, the types of programming tasks used, and the methodology for assessing AI performance in solving AI-generated exercises. The results are described in Section 4, presenting empirical findings on the effectiveness of solving LLM-generated exercises by LLMs. Finally, the Discussion (Section 5) and Conclusion (Section 6) sections provide an analysis of key insights, acknowledge limitations, and outline directions for future research.

2 State of the art review

Recent research has demonstrated the ability of LLMs to generate programming tasks, often involving agents [12, 19] that exhibit high levels of syntactic and semantic fidelity, achieved through their ability to model complex probabilistic relationships between code structures and natural language descriptions. Current models often exploit transformer-based architectures [9], which excel in capturing long-range dependencies and contextual nuances, enabling them to produce exercises that are technically accurate and pedagogically aligned with specific learning objectives [16]. For instance, state-of-the-art frameworks have incorporated fine-tuning strategies using domain-specific corpora, such as annotated repositories of educational coding problems or curated datasets from competitive programming platforms, to enhance task relevance and difficulty calibration. Furthermore, recent innovations in prompt engineering and few-shot learning have enabled LLMs to adapt dynamically to varying instructional contexts [2], including introductory programming courses, advanced algorithm design, and

specialized domains like machine learning and distributed systems. They have even been demonstrated to generate exercises specifically prepared for gamified learning environments such as FGPE [21].

On the evaluation front, contemporary methodologies integrate multimodal assessment techniques that combine static analysis, dynamic testing, and formal verification to ensure robust and comprehensive feedback [23]. Static analysis methods, powered by abstract syntax tree (AST) parsing and control flow graph (CFG) construction, allow LLMs to identify structural anomalies, inefficiencies, and potential vulnerabilities in submitted solutions [4]. On the other hand, dynamic testing frameworks employ generative adversarial networks (GANs) or reinforcement learning (RL) to synthesize diverse and challenging test cases [3], including edge cases and adversarial inputs, thus probing the robustness of algorithms. In particular, recent studies have explored the integration of symbolic execution and SMT solvers to verify correctness properties and infer computational complexity metrics [26], providing learners with precise insight into the theoretical guarantees of their implementations. In addition, explainability-driven approaches [1], such as attention-based visualization and rationale generation, have been introduced to elucidate the reasoning behind the evaluation results, fostering a deeper conceptual understanding among learners [24].

Despite the notable advances in this area, several scientific challenges remain unresolved, necessitating further investigation. One critical limitation is the inherent bias in training data [31], which can often propagate disparities in exercise difficulty, topic coverage, and cultural relevance. Addressing it requires the development of debiasing algorithms [32] and creating more inclusive and representative datasets, potentially through collaborative efforts involving educators, domain experts, and AI researchers. Another area of concern is the stochastic nature of LLM outputs [22], which introduces variability in exercise quality and evaluation consistency. In addition, ethical considerations, such as ensuring equitable access to LLM-powered tools and safeguarding against misuse in academic integrity violations, demand rigorous scrutiny. Ongoing research [15] explores mechanisms for embedding transparency and accountability into these systems, including interpretable model architectures and blockchain-based audit trails. As the field progresses, addressing these challenges remains a priority for many researchers seeking to advance LLMs' scientific understanding and practical utility as transformative tools in computational education. Our study aims to investigate this gap by evaluating how AI behaves in solving AI-generated programming exercises.

3 Materials and Methods

3.1 Selection of Large Language Models

LLMs like ChatGPT, DeepSeek, Qwen, and Gemini have transformed programming education by offering personalized, interactive, and scalable learning experiences. Their underlying architectures and meticulously curated training data empower these models to deliver exceptional performance in coding-related tasks.

Whether helping beginners grasp fundamental programming principles or helping seasoned professionals tackle complex projects, these LLMs continue to push the boundaries of what is possible in programming education.

ChatGPT, developed by OpenAI, is based on the GPT (Generative Pre-trained Transformer) architecture, which uses a transformer-based neural network designed for natural language processing tasks. The model is pre-trained on a vast corpus of text data from the internet, books, and other sources, enabling it to understand and generate human-like responses across diverse topics. ChatGPT has been fine-tuned for programming education to comprehend and generate code in multiple programming languages such as Python, JavaScript, Java, and C++. Its capabilities include explaining complex coding concepts, generating code snippets, debugging errors, and suggesting algorithmic improvements. By engaging with ChatGPT, learners can receive instant feedback on their code, ask questions about programming paradigms, and explore real-world applications of programming techniques.

DeepSeek, another LLM capable of generating programming tasks, employs a similar transformer-based architecture but focuses heavily on technical and coding-related content during its training phase. DeepSeek's training data includes many open-source code repositories, technical documentation, and programming forums, ensuring it profoundly understands coding practices and conventions. This specialization allowed DeepSeek to excel in tasks like code completion, syntax correction, and translating code between different programming languages. In addition, DeepSeek supports advanced features such as generating unit tests, optimizing runtime performance, and offering insights into software design patterns. Its ability to provide context-aware suggestions makes it an invaluable tool for novice programmers learning the basics and experienced developers seeking to refine their skills.

Qwen, developed by Alibaba Cloud, builds upon the transformer architecture with enhancements to improve efficiency and scalability. Qwen's training data cover many sources, including Alibaba's internal historical accumulation, web texts, and specialized data sets focused on technology and programming. It enables Qwen to handle various programming-related queries effectively. In programming education, Qwen demonstrates proficiency in generating high-quality code, explaining intricate algorithms, and assisting with project-based learning. It integrates seamlessly with development environments, offering real-time support through plugins and APIs. Furthermore, Qwen's multilingual capabilities extend its reach to non-English speaking communities, democratizing access to quality programming education worldwide.

Gemini, developed by Google, represents another cutting-edge LLM capable of addressing complex programming challenges. Based on extensive research by Google in machine learning and natural language processing, Gemini uses a sophisticated transformer architecture optimized for handling long-range dependencies and contextual nuances in code. Its training data includes a rich mix of public code repositories, academic papers, and technical manuals, equipping it with a comprehensive understanding of programming languages and frame-

works. Gemini stands out in its ability to perform end-to-end software development tasks, from initial concept ideation to final deployment. It can draft entire programs based on high-level specifications, conduct thorough code reviews, and propose innovative solutions to computational problems. Moreover, Gemini's integration with Google's ecosystem provides users access to powerful tools and resources, enhancing their overall learning experience.

3.2 Dataset and Exercise Generation

Large language models (LLMs) generate programming exercises like those offered by DeepSeek, ChatGPT, Qwen, and Gemini. The methodology used is a structured rapid engineering approach, where the exercise needs are carefully stated to guarantee consistent and pedagogically sound output. From beginner tasks on fundamental concepts (such as conditional structures and loops) to more complex challenges requiring the creation of intricate algorithms or sophisticated data structures, the prompts accommodate varying degrees of difficulty. The process of generating includes:

- Definition of the prompt: this is defined as the exercise's precise and comprehensive articulation, including its goals and limitations;
- Prompt to LLMs: DeepSeek and ChatGPT are used to generate code automatically;
- Output collection: contrasting produced solutions to examine differences in code accuracy, efficiency, and quality.

Exercises are chosen to teach a wide range of skills, ensuring the problems align with the desired learning objectives in the particular educational context.

3.3 Prompt Engineering and Query Structure

A structured prompt engineering strategy was adopted to ensure consistency in exercise generation and code execution. Two main prompts were used: generating exercises (Generator) and executing and solving them (Executor).

Generator Prompt The following prompt was used to generate structured programming exercises in JSON format:

```
Generate a list of 10 programming exercises in JSON format.
↳ Each exercise must strictly follow this structure:
[
  {
    "name": "<A_concise_and_clear_title_for_the_
↳ exercise>",
    "description": "<A_detailed_description_of_the_
↳ exercise ,_specifying_exactly_what_needs_to_be_
↳ implemented>",
```

```

    "input": [<A list of example inputs of the correct
              ↪ type that will be used to test the solution>],
    "output": [<A list of expected outputs of the correct
               ↪ type corresponding to each input>]
  }
]

```

Constraints:

1. Each exercise must explicitly define a *set* of input
↪ *values* in the "input" field.
2. Each exercise must explicitly define the *expected output*
↪ *values* in the "output" field.
3. The *number of elements* in "input" and "output" must
↪ *always match*, ensuring testability.
4. *Data types must be correct for both inputs and outputs*:
 - If the function expects *integers*, provide integers in
↪ "input", not strings.
 - If the function expects *floating-point numbers*, use
↪ decimals (e.g., 3.14).
 - If the function expects *strings*, provide strings.
 - If the function expects *boolean values*, return actual
↪ boolean values (true/false), not strings.
 - If the function expects *lists*, provide lists with
↪ elements of the correct type.
 - If the function expects *dictionaries (maps)*, provide
↪ correctly formatted JSON objects.
5. Ensure diverse exercises covering different programming
↪ concepts such as *loops*, *recursion*, *data structures*,
↪ and *algorithms*.
6. The *JSON output* must be properly formatted and valid.

Executor Prompt The execution prompt ensured the generated code adhered to the expected input-output behavior. The following structured request was issued to the LLMs to execute the exercise, i.e., generate its solution in the form of a source code in the indicated programming language:

```

Return a {self.language} code that MUST take as input some
  ↪ parameters and MUST produce an output based on the
  ↪ given description.
The function MUST return only the result, without any string.
If the exercise has more than one argument, ensure that all
  ↪ parameters are correctly handled and passed to the
  ↪ function.
The function should accept multiple inputs separately and
  ↪ process them accordingly.
If the language is Java, add also the main and read the
  ↪ parameters from args.

```

```

Exercise: {data['name']}
Description: {data['description']}

The response must be in a valid JSON format:
{
    "exercise": "{data['name']}",
    "solution": {
        ↪ json.dumps(self.generate_function_template())
    }
}

```

3.4 Automated Code Evaluation Techniques

Our methodology uses a systematic verification procedure to assess the code produced by LLMs, which consists of the following:

- Operating and contrasting results: the produced code is run in a regulated setting, and the outcomes are contrasted between the DeepSeek and ChatGPT APIs. This stage highlights variations in the models' approaches and enables the identification of any discrepancies in the findings;
- Dynamic testing with test case generation: creating test cases for each exercise that tests the code's behavior under standard input and edge circumstances is a component of dynamic testing. By determining whether the solutions appropriately handle specific scenarios, the objective is to assess the solutions' robustness;
- Qualitative code analysis: qualitative code analysis measures elements like readability, structure, and adherence to best practices for programming in addition to functional correctness. This assessment aids in determining whether one model often generates code that is easier to comprehend and maintain than the other.

This approach allows for ascertaining the generated code's accuracy and how ChatGPT and DeepSeek produce programmatic solutions.

3.5 Experimental Setup and Metrics

The experimental setup aims to systematically assess how well LLMs generate and solve programming problems. Because the studies were conducted locally, complete control over the execution conditions and their repeatability were guaranteed. The study focuses on the syntactical and logical correctness of programming exercise solutions generated by different LLMs.

Included in the test environment are:

- **Controlled Local Execution:** All tests were performed locally to eliminate variability due to network latency or cloud-based execution inconsistencies.
- **Standardized Testing Procedure:** Each LLM was evaluated in multiple phases, alternating between the exercise generator and solver roles. It ensured a balanced comparison among all models.

- **Testing Cycles:** In each evaluation cycle, one model acted as the generator, producing 20 exercises. Each of the remaining three models then solved the exercises. Each exercise was attempted in Python, JavaScript, and Java by solving LLM, ensuring a comprehensive assessment across multiple coding paradigms. This methodology ensured that each model was evaluated relatively in generation and solving tasks, with consistent test cases per language and role.

We defined the following evaluation metrics:

- Err_s : Rate of syntax errors in the generated solutions.
- Err_l : Rate of logical errors in the solutions.
- TS : An overall total score index expressed in percentage as the normalized sum of the logical and syntax errors in automatically generated solutions.

These structured methodologies assess LLM performance in programming tasks, focusing on correctness, efficiency, and robustness in real-world execution scenarios.

4 Results

In this study, we analyze the ability of LLMs to solve programming exercises provided in a structured format, evaluating their correctness through the execution of the generated code.

Table 1 represents the results in terms of the rate of syntax errors in the generated solutions. Each LLM model acts as an exercise generator (on the rows) and as an exercise executor (on the columns) performing the solving process. The first line shows the Err_s obtained by each LLM while ChatGPT is the Generator. The **Mean Generator** column averages the results of each LLM model. Lower values mean the Generator produced exercises that the LLM models straightforwardly solved. In the case of Err_s , ChatGPT-generated exercises are the most suitable for the other LLMs. The **Mean Executors** row evaluates which executor is more capable of solving AI-generated exercises. In this case, all the LLMs perform similarly except for the Qwen model.

Err_s	ChatGPT	DeepSeek	Qwen	Gemini	Mean Generators
ChatGPT	0.043	0.044	0.061	0.045	0.048
DeepSeek	0.056	0.053	0.072	0.061	0.061
Qwen	0.072	0.067	0.068	0.056	0.066
Gemini	0.056	0.061	0.065	0.070	0.063
Mean Executors	0.057	0.056	0.066	0.058	

Table 1. Comparing Err_s

Table 2 represents the results regarding the rate of logical errors in the generated solutions. In the case of Err_l , Gemini-generated exercises are the most prone to generating logical errors, while the other LLMs perform similarly. From the executors' performance point of view, all the LLMs perform equally except for the Qwen model.

Err_l	ChatGPT	DeepSeek	Qwen	Gemini	Mean Generators
ChatGPT	0.005	0.012	0.013	0.011	0.010
DeepSeek	0.006	0.011	0.028	0.006	0.012
Qwen	0.017	0.017	0.016	0.011	0.015
Gemini	0.017	0.022	0.033	0.019	0.023
Mean Executors	0.011	0.015	0.023	0.012	

Table 2. Comparing Err_l

Finally, Table 3 represents the overall total score index summarizing both Err_s and Err_l and then normalizing the result on the total amount of generated exercises. This work uses the TS index for the overall evaluation. Considering the results shown in the table, ChatGPT is the best-performing LLM model for both generating exercises and solving exercises, behaving slightly better as a generator with an error rate of about 6 %. At the same time, as a solver, it has an overall total score index of about 7 %. Gemini is the model that behaves worst in generating exercises. From the exercise executor's point of view, all the models behave in a similar fashion except Qwen, with an overall total score of about 9 %-

%	ChatGPT	DeepSeek	Qwen	Gemini	Mean Generators
ChatGPT	4.8	5.6	7.4	5.6	5.9
DeepSeek	6.1	6.4	10.0	6.7	7.3
Qwen	8.9	8.3	8.3	6.7	8.0
Gemini	7.2	8.3	9.8	8.9	8.6
Mean Executors	6.8	7.2	8.9	7.0	

Table 3. Comparing the overall total score score index (TS).

This analysis is critical for understanding the level of variability in the responses and assessing the model's reliability in educational and automated assessment contexts.

For this analysis, we submitted 20 exercises for each of the three languages available to each LLM Executor and found these points:

1. ChatGPT Generator:

- Python has the lowest rate of syntax errors, making it the most stable language with the least variation between runs.
 - Java has the highest number of unsolved exercises, with the highest rate of syntax errors, showing more significant variability in solutions than Python and JavaScript.
 - The DeepSeek model is the fastest in solving exercises.
 - The Qwen model is the best in terms of solving.
2. DeepSeek Generator:
 - Java continues to have the highest rate of syntactic errors, averaging 5 percent, making it the most difficult language to handle among repeated executions.
 - The GPT model is the fastest to solve the exercises, being more efficient than the other executors in this combination.
 3. Qwen Generator:
 - Python exhibits the most deterministic behavior, with stable syntax errors, while JavaScript exhibits slight variation.
 - The Gemini model is the fastest to solve exercises.
 - The GPT and the DeepSeek models are the best in resolution, achieving a higher percentage of correct exercises than the others.
 4. Gemini Generator:
 - Python and JavaScript continue to be the most stable languages.
 - The Qwen model is the fastest at solving exercises.
 - The GPT model is the best in terms of resolution, having the highest percentage of correct exercises and fewer logical errors than the others.

We have also conducted a permutation-based statistical test to analyze the influence of generator and executor models on the error rates in AI-generated programming exercises. This test was applied separately to three key error metrics: overall total score (TS), syntax errors (Err_s), and logical errors (Err_l) across three programming languages: Python, JavaScript (JS), and Java. The null hypothesis states that observed differences in error rates across different generator-executor pairings arise purely from random variation rather than a systematic effect.

The results, summarized in Table 4, indicate that none of the tested error metrics show a statistically significant difference ($p > 0.05$). For execution errors (Err_{LLME}), the highest observed difference was in Java (7.0), followed by JavaScript (7.0) and Python (2.0). However, permutation tests yielded p-values greater than 0.25, suggesting that any observed variation is likely due to chance. For syntax errors (Err_s), the most significant observed difference was found in JavaScript (5.0), with Python showing a slightly lower difference (3.0). Again, permutation testing indicated these differences are not statistically significant, implying syntax errors occur with similar frequency across different generator-executor configurations. The observed differences for logical errors (Err_l) were lower than for execution and syntax errors, with JavaScript and Java showing slightly larger values than Python. However, as with the other error metrics,

all p-values remained above 0.05, confirming that the generator and executor models do not systematically influence logical error rates.

Error Variable	Language	Observed Difference (%)	Mean Permuted Difference (%)	p-Value
TS	Python	3.33	3.10	0.770
TS	JavaScript	11.67	10.32	0.430
TS	Java	11.67	8.63	0.260
Err_s	Python	5.00	4.00	0.510
Err_s	JavaScript	8.33	7.32	0.500
Err_s	Java	6.67	6.47	0.490
Err_l	Python	1.67	1.60	0.620
Err_l	JavaScript	3.33	2.87	0.540
Err_l	Java	5.00	4.18	0.500

Table 4. Permutation test results for error rates across different programming languages (values as percentages)

5 Discussion

Recent studies have evaluated the efficacy of LLMs in solving programming challenges designed by humans for programming contests and courses [8, 10, 29, 27]. The results of these studies demonstrate that LLMs excel at well-defined tasks but struggle with ambiguous, creative, or complex challenges, as well as producing efficient code. LLMs can generate correct code but often make syntax or logical errors. The quality of the prompts significantly influences this performance. Moreover, ChatGPT (GPT-4) has been the most effective problem solver in these studies, solving about 60% of the programming tasks proposed [10].

This study explores the effectiveness of LLMs in solving programming exercises generated by other LLMs. We have designed two comprehensive prompts, one for generating exercises and another to solve them. With these prompts, we executed four experiments rotating the roles of the LLMs, i.e., in each turn, one LLM generates 20 challenges and the others try to solve them. In line with previous studies, the results demonstrate that the GPT is the model capable of solving more challenges, faster, and with fewer logical errors, independently of the LLM generating the challenge. DeepSeek is second in all measured metrics, except in the ratio of syntax errors, which performs slightly better than GPT. Qwen is the model producing more syntax and logic errors and solving less challenges.

Regarding the quality of the challenges generated by LLMs, we measure it based on the average errors and unsolved challenges by the solvers (i.e., the fewer errors committed by the solvers, the higher the quality of the generated challenges). Once again, ChatGPT led the scoreboard in all metrics, followed by DeepSeek. Gemini was the worst generator if we consider the average number of

unsolved challenges or logical errors that occurred, while challenges generated by Qwen originated more syntax errors in solutions.

Our experiments reveal that LLMs can effectively solve AI-generated exercises with varying degrees of accuracy across different programming languages. The structured approach used in this study allowed us to analyze error patterns, highlighting frequent syntax and logical mistakes, particularly in Java-based solutions. While LLM-based assessments ensure objectivity and consistency, they sometimes misinterpret problem constraints or fail to validate alternative correct answers. These findings reinforce the need for human oversight to refine AI-generated exercises and solutions.

6 Conclusion

This study takes an original approach to investigating the effectiveness of state-of-the-art LLMs in the double role of exercise generators and solvers.

Although the simple comparison of the mean error rates in solutions generated by different LLMs indicates notable differences, the results of the performed statistical analysis show that the difference was not statistically significant for any pair of generator and executor models.

Future research should improve LLMs' understanding of problem requirements and generate pedagogically relevant exercises. Enhancing AI-driven feedback mechanisms to provide explanations beyond correctness will make AI assessments more informative. Refining prompt engineering strategies can lead to better exercise generation and evaluation accuracy. Hybrid AI-human grading systems should be explored further to mitigate AI limitations while leveraging efficiency.

Acknowledgments. This research was co-funded by the European Union, grant number 2023-1-PL01-KA220-HED-000164696.

Disclosure of Interests. The authors have no competing interests to declare relevant to this article's content.

References

1. Abu-Rasheed, H., Abdulsalam, M.H., Weber, C., Fathi, M.: Supporting student decisions on learning recommendations: An llm-based chatbot with knowledge graph contextualization for conversational explainability and mentoring. arXiv preprint arXiv:2401.08517 (2024)
2. Cain, W.: Prompting change: exploring prompt engineering in large language model ai and its potential to transform education. *TechTrends* **68**(1), 47–57 (2024)
3. Chowdhary, A., Jha, K., Zhao, M.: Generative adversarial network (gan)-based autonomous penetration testing for web applications. *Sensors* **23**(18), 8014 (2023)
4. Deng, L., Zhong, Q., Qiu, Y., Chen, J., Lei, H., Yang, S., Zhou, L., Cheng, H.: Llm-based program analysis for source codes, abstract syntax trees and webassembly instructions. ResearchSquare preprint (2024), <https://doi.org/10.21203/rs.3.rs-5419799/v1>

5. Dou, S., Jia, H., Wu, S., Zheng, H., Zhou, W., Wu, M., Chai, M., Fan, J., Huang, C., Tao, Y., et al.: What's wrong with your code generated by large language models? an extensive study. arXiv preprint arXiv:2407.06153 (2024)
6. Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large language models for software engineering: Survey and open problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). pp. 31–53 (2023). <https://doi.org/10.1109/ICSE-FoSE59343.2023.00008>
7. Fan, G., Xie, X., Zheng, X., Liang, Y., Di, P.: Static code analysis in the ai era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents. arXiv preprint arXiv:2310.08837 (2023)
8. Hou, W., Ji, Z.: Comparing large language models and human programmers for generating programming code. *Advanced Science* **n/a**(n/a), 2412279 (2024). <https://doi.org/10.1002/advs.202412279>, <https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/advs.202412279>
9. Huang, D., Yan, C., Li, Q., Peng, X.: From large language models to large multi-modal models: A literature review. *Applied Sciences* **14**(12), 5068 (2024)
10. Huang, Y., Lin, Z., Liu, X., Gong, Y., Lu, S., Lei, F., Liang, Y., Shen, Y., Lin, C., Duan, N., Chen, W.: Competition-level problems are effective llm evaluators (2024), <https://arxiv.org/abs/2312.02143>
11. Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S.: A survey on large language models for code generation (2024). <https://doi.org/10.48550/ARXIV.2406.00515>, <https://arxiv.org/abs/2406.00515>
12. Jin, H., Lee, S., Shin, H., Kim, J.: Teach ai how to code: Using large language models as teachable agents for programming education. In: Proceedings of the CHI Conference on Human Factors in Computing Systems. pp. 1–28 (2024)
13. Joel, S., Wu, J.J., Fard, F.H.: A survey on llm-based code generation for low-resource and domain-specific programming languages (2024). <https://doi.org/10.48550/ARXIV.2410.03981>, <https://arxiv.org/abs/2410.03981>
14. Kumar, P.: Large language models (llms): survey, technical frameworks, and future challenges. *Artificial Intelligence Review* **57**(10), 260 (2024)
15. Lee, J., Hicke, Y., Yu, R., Brooks, C., Kizilcec, R.F.: The life cycle of large language models in education: A framework for understanding sources of bias. *British Journal of Educational Technology* **55**(5), 1982–2002 (2024)
16. Li, Q., Fu, L., Zhang, W., Chen, X., Yu, J., Xia, W., Zhang, W., Tang, R., Yu, Y.: Adapting large language models for education: Foundational capabilities, potentials, and challenges. arXiv preprint arXiv:2401.08664 (2023)
17. Liu, J., Xia, C.S., Wang, Y., ZHANG, L.: Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*. vol. 36, pp. 21558–21572. Curran Associates, Inc. (2023)
18. Lyu, M.R., Ray, B., Roychoudhury, A., Tan, S.H., Thongtanunam, P.: Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* (Dec 2024). <https://doi.org/10.1145/3708519>, <http://dx.doi.org/10.1145/3708519>
19. Ma, Q., Shen, H., Koedinger, K., Wu, S.T.: How to teach programming in the ai era? using llms as a teachable agent for debugging. In: International Conference on Artificial Intelligence in Education. pp. 265–279. Springer (2024)

20. Ma, W., Liu, S., Lin, Z., Wang, W., Hu, Q., Liu, Y., Zhang, C., Nie, L., Li, L., Liu, Y.: Lms: Understanding code syntax and semantics for code analysis. arXiv preprint arXiv:2305.12138 (2023)
21. Montella, R., Giuseppe De Vita, C., Mellone, G., Ciricillo, T., Caramiello, D., Di Luccio, D., Kosta, S., Damasevicius, R., Maskeliunas, R., Queiros, R., Swacha, J.: GAMAI, an AI-powered programming exercise gamifier tool. In: Olney, A.M., Chounta, I.A., Liu, Z., Santos, O.C., Bittencourt, I.I. (eds.) *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium and Blue Sky*. pp. 485–493. Springer Nature Switzerland, Cham, Switzerland (2024). https://doi.org/10.1007/978-3-031-64315-6_47
22. Ouyang, S., Zhang, J.M., Harman, M., Wang, M.: Llm is like a box of chocolates: the non-determinism of chatgpt in code generation. arXiv preprint arXiv:2308.02828 (2023)
23. Possaghi, I., Zhang, F., Sharma, K., Papavlasopoulou, S.: Design thinking activities for k-12 students: Multi-modal data explanations on coding performance. In: *Proceedings of the 23rd Annual ACM Interaction Design and Children Conference*. pp. 290–306 (2024)
24. Song, B., Miller, S., Ahmed, F.: Attention-enhanced multimodal learning for conceptual design evaluations. *Journal of Mechanical Design* **145**(4), 041410 (2023)
25. Stamper, J., Xiao, R., Hou, X.: Enhancing llm-based feedback: Insights from intelligent tutoring systems and the learning sciences. In: *International Conference on Artificial Intelligence in Education*. pp. 32–43. Springer (2024)
26. Steinhöfel, D.: Symbolic execution: foundations, techniques, applications, and future perspectives. In: *The Logic of Software. A Tasting Menu of Formal Methods: Essays Dedicated to Reiner Hähnle on the Occasion of His 60th Birthday*, pp. 446–480. Springer (2022)
27. Svetkin, A.: Testing llms on solving leetcode problems. <https://hackernoon.com/testing-llms-on-solving-leetcode-problems> (may 2024), accessed on Feb 18, 2025
28. Tapalova, O., Zhiyenbayeva, N.: Artificial intelligence in education: Aied for personalised learning pathways. *Electronic Journal of e-Learning* **20**(5), 639–653 (2022)
29. Walker, S.M.: Bigcodebench: A new benchmark for evaluating llms on programming tasks. <https://klu.ai/glossary/bigcodebench-eval> (june 2024), accessed on Feb 18, 2025
30. Wang, T., Zhou, N., Chen, Z.: Enhancing computer programming education with llms: A study on effective prompt engineering for python code generation (2024). <https://doi.org/10.48550/ARXIV.2407.05437>, <https://arxiv.org/abs/2407.05437>
31. Warr, M., Oster, N.J., Isaac, R.: Implicit bias in large language models: Experimental proof and implications for education. *Journal of Research on Technology in Education* pp. 1–24 (2024)
32. Yogarajan, V., Dobbie, G., Keegan, T.T.: Debiasing large language models: research opportunities. *Journal of the Royal Society of New Zealand* **55**(2), 372–395 (2025)
33. Zhang, H., Wu, C., Xie, J., Rubino, F., Graver, S., Kim, C., Carroll, J.M., Cai, J.: When qualitative research meets large language model: Exploring the potential of qualigpt as a tool for qualitative coding. arXiv preprint arXiv:2407.14925 (2024)